

INF-09 Ověřím správnost, najde a opraví případnou chybu v algoritmu, otestuji, odladím a optimalizuji program.

Na test máš 30 minut. Pracuj samostatně, bez pomoci druhých i umělé inteligence. Úlohy 1, 2, 3 a 5 řeš bez počítače. U úloh 4, 6 a 7 smíš program spustit v Pythonu (Colab nebo Thonny) – zapiš ale, co jsi čekal a co se stalo. Odpovědi, opravené programy a záznam testů nahraj do portfolio na Google Disku.

1. Urči typ chyby (syntaktická, běhová, logická): a) `for i in range(10)` bez dvojtečky na konci; b) `věk = int(input())`, uživatel napíše „sedm“; c) obsah kruhu jako `3.14 * r * 2`; d) `print(Prumer)`, když proměnná se jmenuje `prumer`; e) `seznam[5]` v seznamu, který má 5 prvků; f) cyklus `while i < 10`, ve kterém se `i` nikdy nezvětší.
2. Program má spočítat průměrnou teplotu, ale skončil chybou. Na kterém řádku chyba nastala, o jaký typ chyby jde a co je její příčinou? Oprav program.

```
teploty = [18.5, 19.0, 20.5, 21.0]
soucet = 0
for i in range(len(teploty) + 1):
    soucet = soucet + teploty[i]
print(soucet / len(teploty))

Traceback (most recent call last):
  File "mereni.py", line 4, in <module>
    soucet = soucet + teploty[i]
IndexError: list index out of range
```

3. Funkce `trojuhelnik(a, b, c)` má pro délky stran vrátit „neexistuje“, „rovnostranný“, „rovnoramenný“ nebo „obecný“. Navrhni testovací sadu aspoň 7 testů: vstup – očekávaný výstup – co test ověřuje. Nezapomeň na hraniční a nesmyslné vstupy.
4. Takhle funkci z úlohy 3 napsal spolužák. Co program vypíše? Které testy neprojdou? Najdi obě chyby a oprav funkci tak, aby prošly všechny testy z tvé sady.

```
def trojuhelnik(a, b, c):
    if a + b <= c or a + c <= b:
        return "neexistuje"
    if a == b == c:
        return "rovnostranný"
    if a == b or b == c:
        return "rovnoramenný"
    return "obecný"

for strany in [(5, 1, 1), (3, 4, 3), (3, 4, 5), (2, 2, 2), (1, 2, 3)]:
    print(strany, trojuhelnik(*strany))
```

5. Krokuj ručně druhé volání funkce: zapiš do trasovací tabulky hodnoty `x` a `nej` po každém průchodu cyklem. Co program vypíše? Jaká je chyba a jak ji opravíš? Jak se má funkce zachovat pro prázdný seznam?

```
def maximum(s):
    nej = 0
    for x in s:
        if x > nej:
            nej = x
    return nej

print(maximum([3, 7, 2]))
print(maximum([-4, -2, -9]))
```

6. Funkce zjistí, jestli se v seznamu opakuje nějaká hodnota. Kolikrát se nejvýš provede vnitřní cyklus pro seznam 1 000 různých čísel? Optimalizuj funkci aspoň jedním způsobem, spočítej, kolik kroků ušetříš, a napiš, jak ověříš, že optimalizovaná verze dává stejné výsledky.

```
def ma_duplicitu(s):
    for i in range(len(s)):
        for j in range(len(s)):
            if i != j and s[i] == s[j]:
                return True
    return False

print(ma_duplicitu([4, 8, 15, 16, 23, 42]))
print(ma_duplicitu([4, 8, 15, 8]))
```

7. Napiš aspoň čtyři příkazy assert, které otestují opravenou funkci maximum z úlohy 5. Zahrň obyčejný případ, samá záporná čísla, jednoprvkový a prázdný seznam.

Test slouží k doložení výstupu do portfolia. Nehodnotí se body ani známkou – rozhoduje, jestli splníš kritéria na konci dokumentu.

Klíč a řešení

1. a) syntaktická; b) běhová (ValueError); c) logická (správně πr^2 , tj. $3.14 * r ** 2$); d) běhová (NameError); e) běhová (IndexError – indexy jsou 0 až 4); f) logická (nekonečný cyklus, program nespadne, ale neskončí).
– ověřuje: rozlišení typů chyb v programu
2. Řádek 4, IndexError (běhová chyba): cyklus běží pro $i = 0$ až 4, ale seznam má indexy jen 0 až 3 – příčina je v řádku 3 (`range(len(teploty) + 1)`). Oprava: `for i in range(len(teploty))`: (nebo `for t in teploty`: `soucet += t`, případně `sum(teploty)`). Opravený program vypíše 19.75. – ověřuje: čtení chybového hlášení, nalezení příčiny chyby a její oprava
3. Např.: (2, 2, 2) rovnostranný; (3, 3, 5) rovnoramenný, $a = b$; (5, 3, 3) rovnoramenný, $b = c$; (3, 5, 3) rovnoramenný, $a = c$; (3, 4, 5) obecný; (1, 2, 3) neexistuje – hranice $a + b = c$; (5, 1, 1) neexistuje – nejdelší strana je první; (0, 1, 1) nebo (-3, 4, 5) neexistuje – nekladná délka. – ověřuje: návrh testovací sady pokrývající všechny větve, hraniční i nesmyslné vstupy
4. Vypiše: (5, 1, 1) rovnoramenný; (3, 4, 3) obecný; (3, 4, 5) obecný; (2, 2, 2) rovnostranný; (1, 2, 3) neexistuje. Neprojdou (5, 1, 1) – má být neexistuje – a (3, 4, 3) – má být rovnoramenný. Chyby: chybí podmínka $b + c \leq a$ a v rovnoramenném chybí $a == c$. Oprava: `if a + b <= c or a + c <= b or b + c <= a: ... a if a == b or b == c or a == c: ...`; navíc je vhodné na začátku odmítnout nekladné délky. – ověřuje: nalezení logické chyby pomocí testů a její oprava
 1. (5, 1, 1): $5 + 1 > 1$ a $5 + 1 > 1$, takže první podmínka neplatí; $b = c \rightarrow$ „rovnoramenný“, ale $1 + 1 < 5$, trojúhelník neexistuje.
 2. Chybí tedy třetí trojúhelníková nerovnost: $b + c \leq a$.
 3. (3, 4, 3): $a = c$, ale funkce testuje jen $a == b$ a $b == c \rightarrow$ „obecný“.
 4. Doplň $a == c$ do podmínky rovnoramenného.
 5. Po opravě spusť znovu celou sadu (i testy, které předtím procházely).
5. Vypiše 7 a 0. Trasování pro [-4, -2, -9]: nej = 0; $x = -4$ (nej 0), $x = -2$ (nej 0), $x = -9$ (nej 0) $\rightarrow$ vrátí 0, ačkoli 0 v seznamu není (správně -2). Chyba: počáteční hodnota 0 – pro samé záporné hodnoty se nej nikdy nezmění. Oprava: `nej = s[0]`. Prázdný seznam: `s[0]` by vyvolal IndexError – ošetřit na začátku (vrátit None nebo vypsát upozornění). – ověřuje: krokování programu a odhalení logické chyby na hraničním vstupu
 1. Začátek: `nej = 0`.
 2. $x = -4$: $-4 > 0$ neplatí, `nej = 0`.
 3. $x = -2$: $-2 > 0$ neplatí, `nej = 0`.
 4. $x = -9$: neplatí, funkce vrátí 0.
 5. Oprava: začínat prvním prvkem (`nej = s[0]`), prázdný seznam ošetřit zvlášť.
6. Původně $1\,000 \cdot 1\,000 = 1\,000\,000$ průchodů vnitřním cyklem. Vylepšení 1: `for j in range(i + 1, len(s))` – každou dvojici porovná jen jednou, bez podmínky $i \neq j$: $\frac{1\,000 \cdot 999}{2} = 499\,500$ porovnání (asi poloviční). Vylepšení 2: `return len(set(s)) != len(s)` – řádově jen 1 000 kroků. Ověření: stejná sada testů pro obě verze (seznam bez opakování, s opakováním na začátku i na konci, prázdný a jednoprvkový seznam, případně mnoho náhodných seznamů porovnaných s původní verzí). – ověřuje: optimalizace programu a ověření, že se nezměnily výsledky
 1. Pro n prvků bez opakování proběhne vnitřní cyklus $n \cdot n$ krát: $1\,000^2 = 10^6$.
 2. Dvojice (i, j) a (j, i) se porovnávají dvakrát a $i = j$ zbytečně – stačí j od $i + 1$.
 3. Počet porovnání: $(n - 1) + (n - 2) + \dots + 1 = \frac{n(n - 1)}{2} = 499\,500$.
 4. S množinou: `set` odstraní opakování, stačí porovnat délky.
 5. Po úpravě spusť stejné testy jako před ní – výsledky se musí shodovat.
7. Např.: `assert maximum([3, 7, 2]) == 7`; `assert maximum([-4, -2, -9]) == -2`; `assert maximum([5]) == 5`; `assert maximum([]) is None` (podle toho, jak jsi prázdný seznam ošetřil). Původní verze neprojde druhým testem.
– ověřuje: zápis automatických testů včetně hraničních případů

Kritéria splnění výstupu

- Správně určí typ chyby aspoň v 5 z 6 případů (úloha 1) a z chybového hlášení určí řádek, typ i skutečnou příčinu chyby (úloha 2).
- Navrhne testovací sadu, která pokryje všechny větve i hranice – rovnost $a + b = c$, rovnost každé dvojice stran a nekladnou délku (úloha 3).

- Pomocí testů nebo krokování najde a opraví logické chyby v úlohách 4 a 5 a opravu ověří znovu spuštěnými testy (úloha 7).
- Optimalizuje program z úlohy 6 aspoň jedním způsobem, správně odhadne počet kroků před a po úpravě a popíše, jak ověří, že se výsledky nezměnily.

Pro učitele: varianty a přizpůsobení

Obměny: v úloze 3–4 jiná funkce s několika větvemi (vstupné podle věku, klasifikace BMI, výpočet poštovného); v úloze 5 hledání minima s počáteční hodnotou 100; v úloze 6 hledání dvojice se zadaným součtem. Silnější žáci mohou v úloze 6 změřit čas obou verzí modulem time pro 1 000, 2 000 a 4 000 prvků. Slabším lze v úloze 3 dát kostru tabulky testů (vstup – očekávaný výstup – co ověřuje). Úlohu 5 lze dělat v Python Tutor nebo v debuggeru Thonny se sdílenou obrazovkou v Meetu. Kratší verze na 15 minut: úlohy 2, 4 a 5.

Výstup se probírá v lekcích: 1.+2. H47 Testování a ladění (týden 24); 1.+2. H63 Programátorský projekt (týden 32)